



PATIENT APPOINTMENT SCHEDULING SOFTWARE



SUBMITTED BY :
SUMIT RANJAN SATAPATHY
2001109212 (3rd SEM , CSE)
swadeshsatapathy90@gmail.com



INTRODUCTION



Patient Appointment Scheduling Software (PASS) , also known as Meeting Scheduling Tools Software (MSTs) , allow businesses & professionals to manage appointments & bookings . This type of software is also known as Appointment Booking Software or Online Booking Software .

Before in the Health Sector Bodies (HSB) , as there is no scope for scheduling appointments because of which lakhs of Patients get into the hospitals per day , due to which the limit of the hospitals get overflowed . On the part of hospital workers such as Doctors & Nurses , it becomes a burdensome job for giving proper treatment to such a huge mass of Patients on a regular basis .



This Patient Appointment Scheduling Software (PASS) will be developed to ease the processes & give both hospitals & Patients the opportunity to experience a convenient schedule .

Like the short form , it also easily passes the hectic task of both Patients & hospital workers , by arranging the appointment date of the Patient , which on the part help both the parties do their tasks efficiently without getting burdened & exhausted . Patients on the part , don't have to wait in long queues , & can come in their scheduled date & time , because of which the limit of the hospital on the basis of the amount of Patients that the hospital workers such as Doctors & nurses , who can give efficient & proper treatment to the pateints , doesn't get overflown .



PLAN OF ACTION



This project based on some of the core concepts of Data Structure & Algorithm (DSA) , that adds the ability to scheduling Patients for appointments , then managing their flow through a clinic , once they arrive for appointments . In simple terms , this project allows implementations to do the following :

- Create a schedule of when providers are available to see Patients .
- Schedule appointments for Patients based on provider schedule .
- Manage the Patient queue through the clinic when the Patients arrive for their appointments .



PROGRAMMING LANGUAGE APPLIED :

➤ C++

INTEGRATED DEVELOPMENT ENVIRONMENT (IDE) APPLIED :

➤ DEV C/C++

CONCEPTS OF DATA STRUCTURE & ALGORITHM (DSA) APPLIED :

➤ ALGORITHM , FLOWCHART , LINEAR SEARCH ALGORITHM (LSA) , INSERTION SORT ALGORITHM (ISA) , SEQUENTIAL SORT ALGORITHM (SSA) , QUEUE , LINKED LIST .



EXECUTION PROCEDURE



SORT ALGORITHM



This sorting algorithm deployed in this Patient Appointment Scheduling Software (PASS) , applied the concept of Insertion Sort Algorithm (ISA) . This sorting algorithm was deployed because the data to be sorted is not much & thus simple to sort . This sorting algorithm was deployed to sort Patient's name in ascending order .



PROGRAMMING SOURCE CODE IMPLEMENTATION OF THE SORT ALGORITHM

```
void insertionSort( )
{
Patient *sorted = NULL ;
Patient *current = head ;
while( current != NULL )
{
Patient *next = current -> next ;
sortedInsert( &sorted , current ) ;
current = next ;
}
```



```
head = sorted ;  
}  
void sortedInsert( Patient **head_ref , Patient *new_node )  
{  
    Patient *current ;  
    if( *head_ref == NULL || ( *head_ref ) -> getFirstName( )  
    >= new_node -> getFirstName( ) )  
    {  
        new_node -> next = *head_ref ;  
        *head_ref = new_node ;  
    }
```



```
}  
else  
{  
current = *head_ref;  
while( ( current -> next != NULL ) && ( ( current -> next  
-> getFirstName( ) ) < ( new_node->getFirstName( ) ) ) )  
{  
current = current -> next ;  
}  
new_node -> next = current -> next ;  
current -> next = new_node ;  
}  
}
```



SEARCH ALGORITHM



This searching algorithm deployed in this Patient Appointment Scheduling Software (PASS) , applied the concept of Sequential Sort Algorithm (SSA) . It is also referred to as the Linear Search Algorithm (LSA) , which starts the comparison from the first entry in the List to the last entry . Upon finding the key item , the result is displayed or else it will keep searching till the last entry . This searching algorithm was deployed for the task of searching of Doctors by the Patients & the searching of Patients through Name & Id number by the Doctors .

.



PROGRAMMING SOURCE CODE IMPLEMENTATION OF THE SEARCH ALGORITHM IN THE MAIN PROGRAM

case 3 :

```
int pos1 ;  
int pos2 ;  
if( user == isPatient )  
{  
    cout << "\n" << "Please enter the doctor first  
name : " ;  
    cin >> firstName ;  
    Sleep( 500 ) ;  
    doctorTableHeaders( ) ;
```



```
doctor.findByFirstName( firstName ) ;  
cout << "-----  
-----  
-----\n\n" ;  
Sleep( 2000 ) ;  
}  
if( user == isDoctor )  
{  
cout << "\n\n" << "Enter Patient ID : " ;  
cin >> id ;  
Sleep( 500 ) ;  
patientInfoTableHeaders( ) ;
```



```
patient.searchPatientInfoById( id ) ;
```

```
cout << "-----
```

```
-----
```

```
-----\n\n" ;
```

```
    Sleep( 2000 ) ;
```

```
    }
```

```
    if( user == isReceptionist )
```

```
    {
```

```
        cout << "\nPlesae enter the doctor first name : " ;
```

```
        cin >> firstName ;
```

```
        Sleep( 500 ) ;
```



```
doctorTableHeaders( ) ;  
doctor.searchDoctorInfoByName( firstName ) ;  
cout << "-----  
-----  
-----\n\n" ;  
Sleep( 2000 ) ;  
}  
break ;
```



PROGRAMMING SOURCE CODE IMPLEMENTATION OF THE SEQUENTIAL SEARCH ALGORITHM (SSA) WITH LINKED LIST

```
void searchDoctorInfoByName( string firstName )
{
    Doctor *currNode = head ;
    while( currNode && currNode -> getFirstName( ) !=
    firstName )
    {
        currNode = currNode -> next ;
    }
}
```



```
if( currNode )
{
currNode->printDoctorInfo( ) ;
}
else
{
cout << "SORRY , THE RECORD IS NOT FOUND "
<< endl ;
}
}
```



```
void displayList( )
{
    Doctor *currNode = head ;
    if( currNode == NULL )
    {
        cout << "Empty \n";
    }
    while( currNode )
    {
        currNode -> printDoctorInfo( ) ;
        currNode = currNode -> next ;
    }
}
```



LINKED LIST IMPLEMENTATION



The concept of Linked List is often implemented in this Patient Appointment Scheduling Software (PASS) , in order to create the Lists of users which are Patients , Doctors & Receptionists . This concept of Linked List is established in the functions which included the boolean `bool isEmpty()` , `findnode()` , `insertnode()` etc .

Let's know the working of every functions used in the programming source code for the making & successful execution of this software .



bool isEmpty() :

The boolean function (bool isEmpty()) returns a null value if the list is empty . This function used to check the list for any entries .



PROGRAMMING SOURCE CODE

```
bool isEmpty( )  
{  
    return ( head == NULL ) ;  
}
```



findNode() :

The function (findNode()) finds a particular node using a key (id) that is a unique identifier of each entry . Upon finding the node , it returns the value .



PROGRAMMING SOURCE CODE FOR UPDATING DOCTOR'S AVAILABILITY STATUS

```
Doctor *findNode( string id , int &index )
{
    Doctor *currNode = head ;
    int size = 0 ;
    while( currNode && size < 10 )
    {
        if( currNode -> schedule[ size ] != NULL )
        {
            if( currNode -> schedule[ size ] -> getScheduleId( ) == id )
```



```
{  
if( currNode -> schedule[ size ] -> getStatus( ) ==  
"available" )  
{  
index = size ;  
currNode -> schedule[ size ] -> setStatus ( " Booked " ) ;  
return currNode ;  
}
```



```
}  
size++;  
}  
else  
{  
size = 0 ;  
currNode = currNode -> next ;  
}  
}  
return NULL ;  
}.
```



PROGRAMMING SOURCE CODE FOR UPDATING PHONE NO. OF PATIENTS

```
void *findNode( string id , string phone )  
{  
    Patient *currNode = head ;  
    while( currNode && currNode -> getId( ) != id )  
    {  
        currNode = currNode -> next ;  
    }
```



```
if( currNode )
{
currNode -> setPhoneNumber( phone ) ;
cout << "\nPHONE NUMBER IS UPDATED
SUCCESSFULLY\n" ;
}
else
{
cout << "SORRY , THE RECORD IS NOT FOUND" ;
}
}
```



PROGRAMMING SOURCE CODE FOR UPDATING PHONE NO. OF PATIENTS

```
Schedule *findNode( string id )
{
    Schedule *currNode = head ;
    while( currNode )
    {
        if( currNode -> getScheduleId( ) != id )
        {
            currNode = currNode -> next ;
        }
    }
}
```



```
else
{
return currNode ;
}
}
return 0 ;
}
```



InsertNode() :

The function (InsertNode()) is used to insert a new node into the List . It Inserts a new node & links the node appropriately to the other nodes in the List .



PROGRAMMING SOURCE CODE FOR CREATING APPOINTMENT OF PATIENTS WITH DOCTORS

```
void InsertNode( Patient *patient , Doctor *doctor ,  
Schedule *schedule )  
{  
Appointment *currNode = head ;  
Appointment *prevNode = NULL ;  
while( currNode && schedule -> getScheduleId( ) >  
currNode -> getAppointmentid( ) )  
{  
prevNode = currNode ;  
currNode = currNode -> next ;  
}
```



```
Appointment *newNode = new Appointment ;  
newNode -> update( patient , doctor , schedule ) ;  
if( prevNode == NULL )  
{  
    newNode -> next = head ;  
    head = newNode ;  
}  
else  
{  
    newNode -> next = prevNode -> next ;  
    prevNode -> next = newNode ;  
}  
}
```



PROGRAMMING SOURCE CODE FOR INSERTION OF INFORMATION OF DOCTORS FROM FILE

```
void InsertNode( List schduel , int i , string doctorId ,
string firstName , string lastName , string gender , string
dateOfBirth , string phoneNumber )
{
int currentIndex = 0 ;
Doctor *currNode = head ;
Doctor *prevNode = NULL ;
while( currNode && doctorId > currNode -> getId( ) )
{
prevNode = currNode ;
```



```
currNode = currNode -> next ;  
}  
Doctor *newNode = new Doctor ;  
newNode -> update( doctorId , firstName , lastName ,  
gender , dateOfBirth , phoneNumber ) ;  
string scId ;  
int index ;  
Schedule *temp ;  
while( i != 0 )  
{  
temp = schduel.getId( ) ;  
if( temp != NULL )  
{
```



```
scId = temp -> getSchedId( ) ;  
index = scId.find( "_" ) ;  
scId = scId.substr( 0 , index ) ;  
if( newNode )  
{  
if( newNode -> getId( ) == scId )  
{  
newNode -> updateSchdual( temp ) ;  
}  
}  
}  
i-- ;  
}
```



```
if( prevNode == NULL )
{
    newNode -> next = head ;
    head = newNode ;
}
else
{
    newNode -> next = prevNode -> next ;
    prevNode -> next = newNode ;
}
}
```



PROGRAMMING SOURCE CODE FOR INSERTION OF A PATIENT RECORD

```
void InsertNode( string patientId , string firstName ,  
string lastName , string gender , string dateOfBirth ,  
string phoneNumber )  
{  
    int currentIndex = 0 ;  
    Patient *currNode = head ;  
    Patient *prevNode = NULL ;  
    while( currNode )  
    {
```



```
prevNode = currNode ;  
currNode = currNode -> next ;  
}  
Patient *newNode = new Patient ;  
newNode -> update( patientId , firstName , lastName ,  
gender , dateOfBirth , phoneNumber ) ;  
if( prevNode == NULL )  
{  
newNode -> next = head ;  
head = newNode ;  
}
```



```
else
{
newNode -> next = prevNode -> next ;
prevNode -> next = newNode ;
}
}
```



PROGRAMMING SOURCE CODE FOR CREATING SCHEDULE OF PATIENTS WITH DOCTORS

```
void InsertNode( string scheduleId , string scheduleDay ,
string scheduleTimeStart , string scheduleTimeEnd , string
status )
{
int currentIndex = 0 ;
Schedule *currNode = head ;
Schedule *prevNode = NULL ;
while( currNode && scheduleId > currNode ->
getScheduleId( ) )
{
```



```
prevNode = currNode ;  
currNode = currNode -> next ;  
}  
Schedule *newNode = new Schedule ;  
newNode -> update( scheduleId , scheduleDay ,  
scheduleTimeStart , scheduleTimeEnd , status ) ;  
if( prevNode == NULL )  
{  
newNode -> next = head ;  
head = newNode ;  
}
```



```
else
{
newNode -> next = prevNode -> next ;
prevNode -> next = newNode ;
}
}
```



`findByFirstName( ) :`

The function (`findByFirstName( )`) implements the Linked List to search for a Patient entry in the system using the Patient's first name & also search for a Doctor in the hospital using the Doctor's first name . It returns the information of the Patient or Doctor , if the node is found in the list or else it returns the statement “ SORRY , THE RECORD IS NOT FOUND ” .



PROGRAMMING SOURCE CODE FOR SEARCHING DOCTOR OR PATIENT BY THEIR FIRST NAME

```
void findByFirstName( string firstName )
{
    Doctor *currNode = head ;
    while( currNode && currNode -> getFirstName( ) != firstName )
    {
        currNode = currNode -> next ;
    }
    if( currNode )
        currNode -> printDoctorSchdual( ) ;
    else
    {
        cout << "Sorry the record not found " << endl;
    }
}
```



deleteNode() :

The function (deleteNode()) is used to delete a node from the list . This function makes use of a key to search for a particular node & deletes that node upon finding it . Afterwhich the remaining nodes in the list are linked immediately .



PROGRAMMING SOURCE CODE FOR THE DELETION OF A PATIENT RECORD

```
int deleteNode( string id )
{
    Patient *prevNode = NULL ;
    Patient *currNode = head ;
    int currIndex = 1 ;
    while( currNode && currNode -> getId( ) != id )
    {
        prevNode = currNode ;
```



```
currNode = currNode -> next ;  
currIndex++ ;  
}  
if( currNode )  
{  
if( prevNode )  
{  
prevNode -> next = currNode -> next ;  
delete currNode ;  
cout << "RECORD DELETED SUCCESSFULLY"  
<< endl ;  
}}
```



```
else
{
head = currNode -> next ;
delete currNode ;
cout << "RECORD DELETED SUCCESSFULLY" <<
endl ;
}
return currIndex ;
}
cout << " SORRY , THE RECORD IS NOT FOUND "
<< endl ;
return 0 ;
}
```



QUEUE IMPLEMENTATION



PROGRAMMING SOURCE CODE IMPLEMENTATION OF THE INSERTION OF A PATIENT RECORD WITH QUEUE

```
bool isEmpty( )  
{  
    return ( head == NULL ) ;  
}  
void InsertNode( string patientId , string firstName ,  
string lastName , string gender , string dateOfBirth ,  
string phoneNumber )  
{  
    int currentIndex = 0 ;
```



```
Patient *currNode = head ;  
Patient *prevNode = NULL ;  
while( currNode )  
{  
    prevNode = currNode ;  
    currNode = currNode -> next ;  
}  
Patient *newNode = new Patient ;  
newNode -> update( patientId , firstName , lastName ,  
gender , dateOfBirth , phoneNumber ) ;
```



```
if( prevNode == NULL )  
{  
    newNode -> next = head ;  
    head = newNode ;  
}  
else  
{  
    newNode -> next = prevNode -> next ;  
    prevNode -> next = newNode ;  
}  
}
```



PROGRAMMING SOURCE CODE IMPLEMENTATION OF THE DELETION OF A PATIENT RECORD WITH QUEUE

```
int deleteNode( string id )
{
    Patient *prevNode = NULL ;
    Patient *currNode = head ;
    int currIndex = 1 ;
    while( currNode && currNode -> getId( ) != id )
    {
        prevNode = currNode ;
```



```
currNode = currNode -> next ;  
currIndex++ ;  
}  
if( currNode )  
{  
    if( prevNode )  
    {  
        prevNode -> next = currNode -> next ;  
        delete currNode ;  
        cout << "RECORD DELETED SUCCESSFULLY"  
        << endl ;  
    }  
}
```



```
else
{
head = currNode -> next ;
delete currNode ;
cout << "RECORD DELETED SUCCESSFULLY"
<< endl ;
}
return currIndex ;
}
cout << "RECORD IS NOT FOUND" << endl ;
return 0 ;
}
```



PROGRAMMING SOURCE CODE IMPLEMENTATION OF THE DISPLAYING OF ALL PATIENT RECORD WITH QUEUE

```
void displayList( )  
{  
    Patient *currNode = head ;  
    if( currNode == NULL )  
    {  
        cout << "EMPTY\n" ;  
    }  
}
```



```
while( currNode )  
{  
currNode -> printPatientList( ) ;  
currNode = currNode -> next ;  
}  
}
```

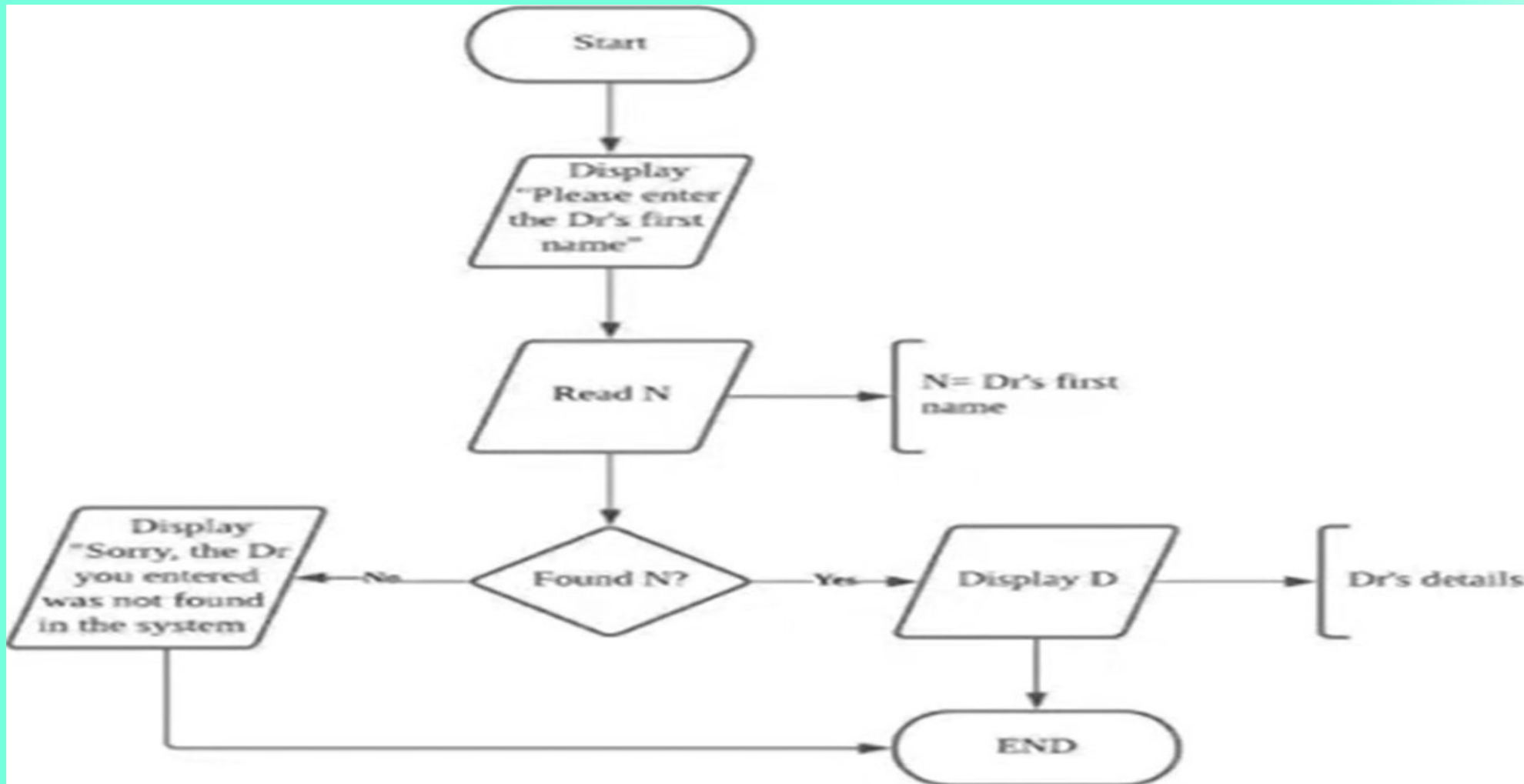


ALGORITHMS WITH FLOWCHART

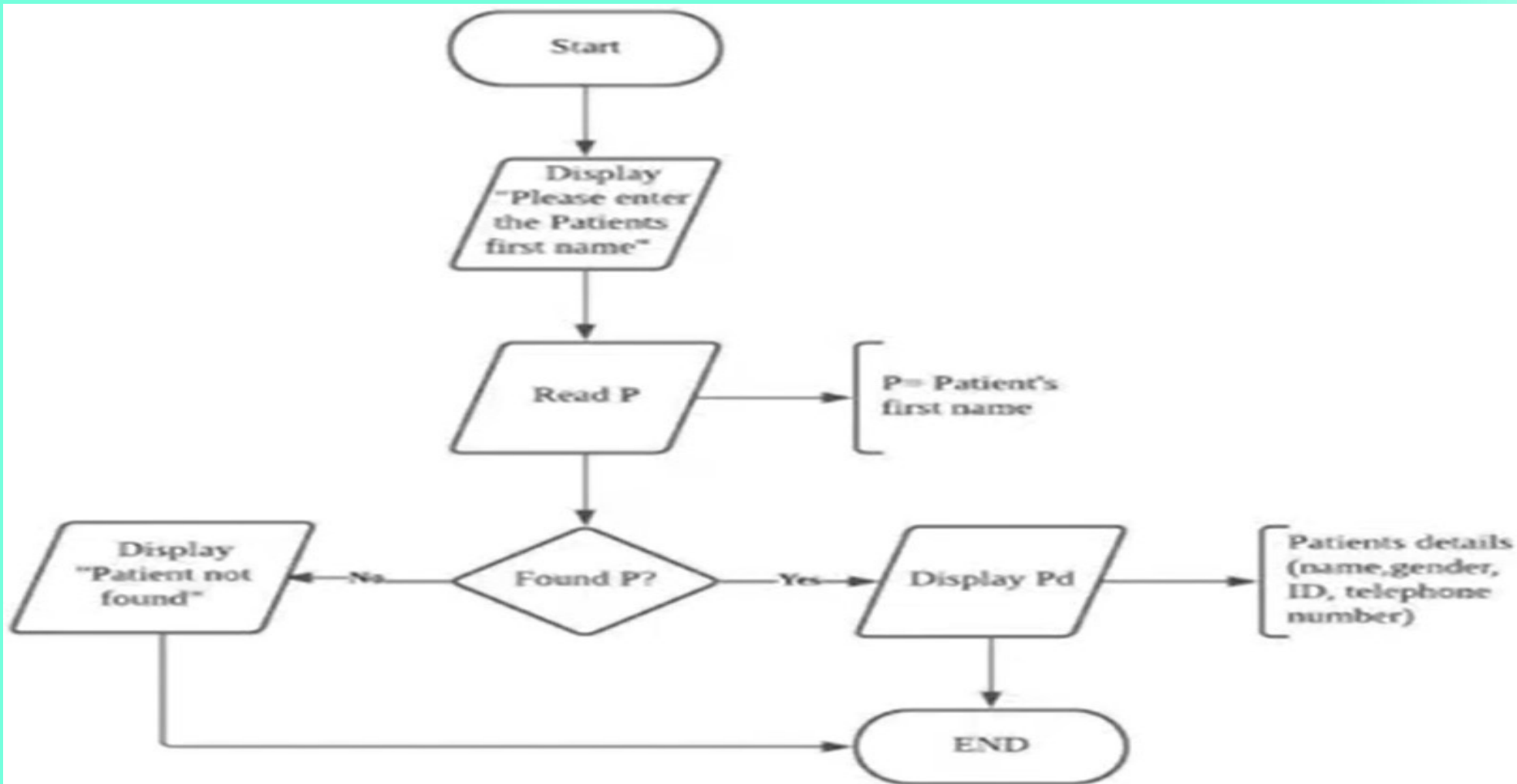
MAKING AN APPOINTMENT FOR ANY PATIENT IN THE HOSPITAL



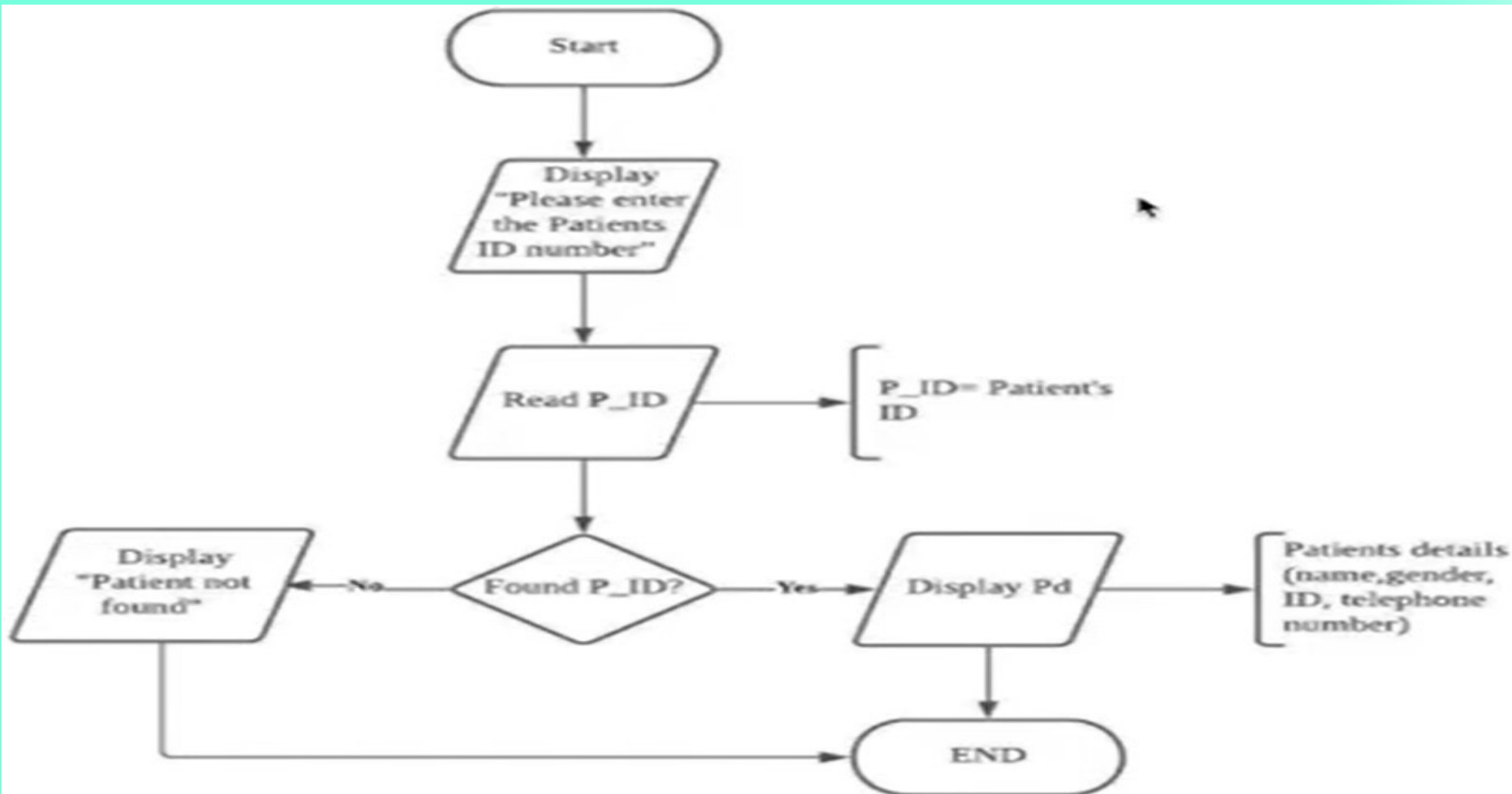
SEARCHING A SPECIFIC DOCTOR BY IT'S FIRST NAME



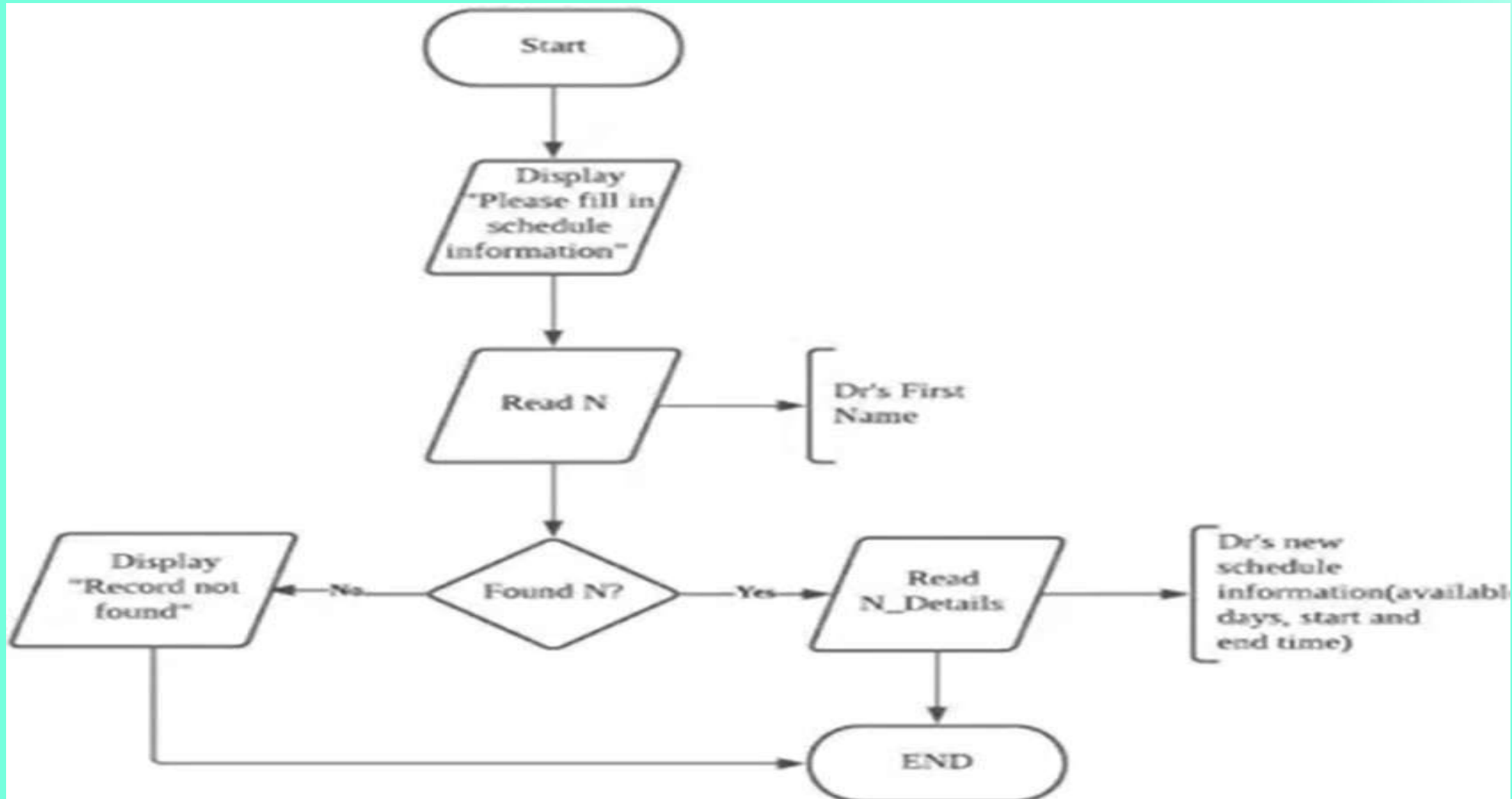
SEARCHING A PARTICULAR PATIENT BY IT'S FIRST NAME



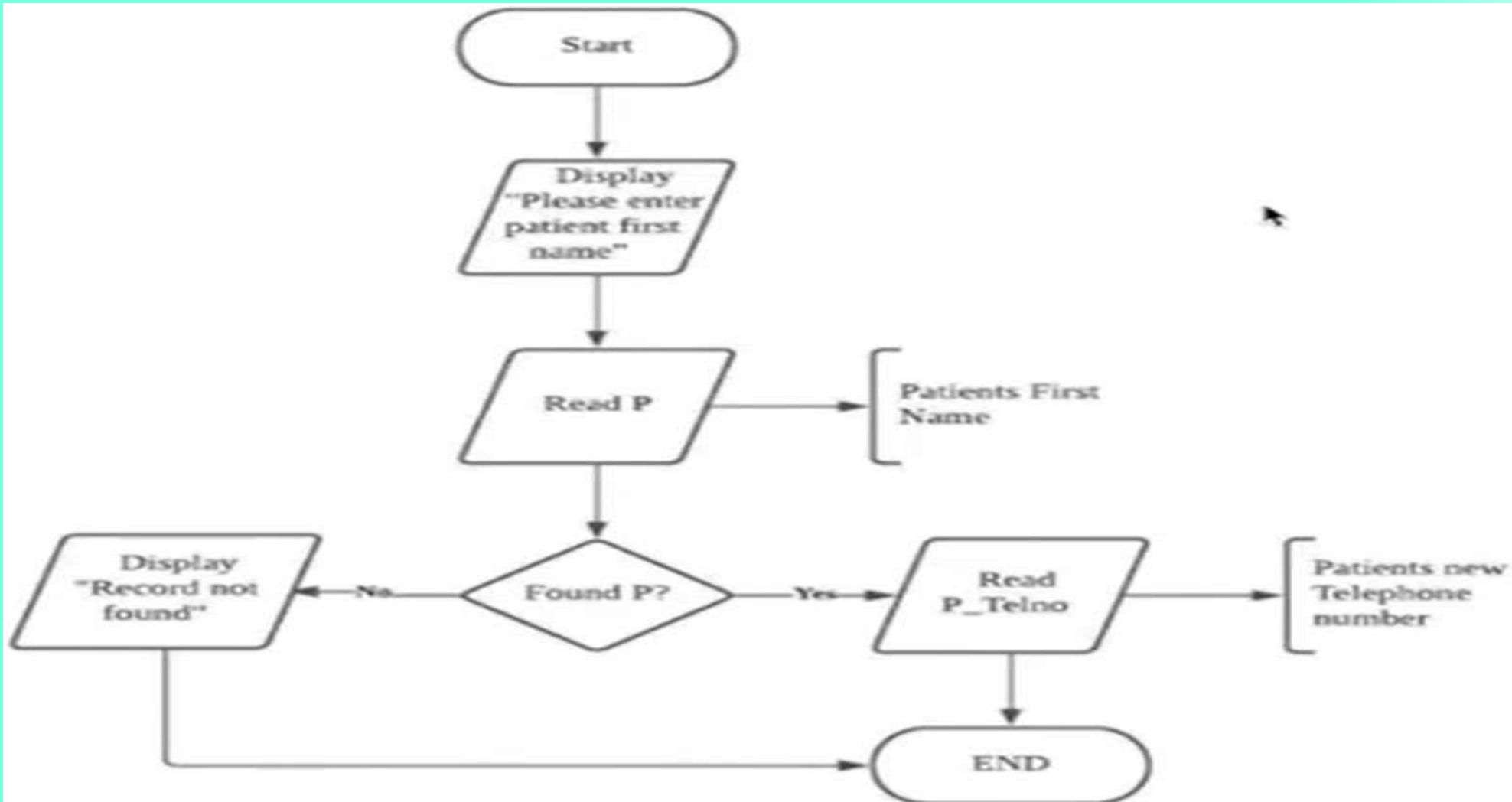
SEARCHING A PARTICULAR PATIENT BY IT'S ID



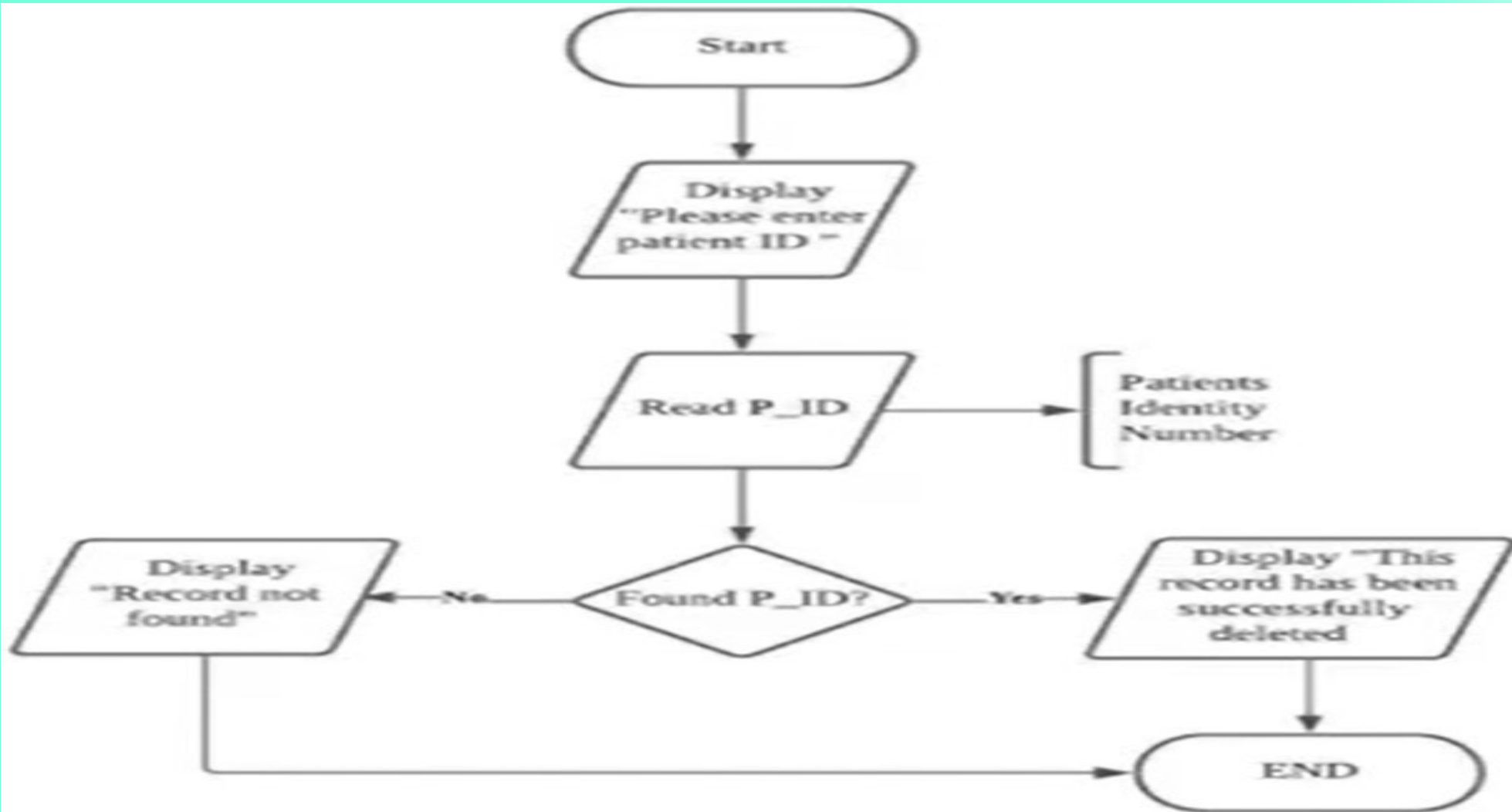
UPDATING SCHEDULE OF ANY DOCTOR IN THE HOSPITAL



UPDATING THE PHONE NUMBER OF ANY PATIENT



DELETING THE RECORD OF ANY PATIENT IN THE HOSPITAL





SOFTWARE PROTOTYPE [OUTPUTS]



PATIENT INTERFACE



USER INTERFACE FOR ANY PATIENT IN THE HOSPITAL

```
*****  
***PATIENT APPOINTMENT SCHEDULING SYSTEM***  
*****
```

```
[ 1 ] PATIENT  
[ 2 ] DOCTOR  
[ 3 ] RECEPTIONIST  
[ 4 ] EXIT
```

```
*****  
Enter your choice :
```



CHOICES AVAILABLE TO ANY PATIENT IN THE HOSPITAL

Enter your choice : 1

- [1] MAKE AN APPOINTMENT
- [2] DISPLAY DOCTORS SCHEDULE
- [3] SEARCH DOCTOR NAME
- [4] SEARCH AVAILABLE DOCTORS
- [8] RETURN TO USER INTERFACE
- [9] EXIT



MAKING AN APPOINTMENT FOR ANY PATIENT IN THE HOSPITAL

What would you like to do? : 1

These are the available date and doctors

DOCTOR NAME	SCHEDULE ID	DAY	START TIME	END TIME	STATUS
ADYASHA TIWARY	mem101184_M1112	MONDAY	11:am	12:am	available
ADYASHA TIWARY	mem101184_M0304	MONDAY	03:pm	04:pm	available
ADITYA MISHRA	tcm11064_T1011	TUESDAY	10:am	11:am	available

Please write the appointment id : mem101184_M0304

Enter First name : SUMIT

Enter Last Name : SATAPATHY

Enter your gender in Capital letter format (MALE / FEMALE) : MALE

Enter your date of birth in format (dd-mm-yyyy) : 23-11-2001

Enter your phone no. : 7978442590

BOOKED SUCCESSFULLY

YOUR BOOKING INFORMATION

PATIENT'S ID	PATIENT'S NAME	DOCTOR'S NAME	APPOINTMENT DAY	APPOINTMENT TIME
PSSM79	SUMIT	ADYASHA	MONDAY	03:pm to 04:pm



DISPLAYING THE WHOLE SCHEDULE OF ALL DOCTORS IN THE HOSPITAL

What would you like to do? : 2

These are the available date and doctors

DOCTOR NAME	SCHEDULE ID	DAY	START TIME	END TIME	STATUS
ADYASHA TIWARY	mem101184_M1112	MONDAY	11:am	12:am	available
ADYASHA TIWARY	mem101184_M0304	MONDAY	03:pm	04:pm	Booked
ADITYA MISHRA	tcm11064_T1011	TUESDAY	10:am	11:am	available



SERACHING A SPECIFIC DOCTOR BY IT'S FIRST NAME

What would you like to do? : 3

Please enter the doctor first name : ADITYA

These are the avilable date and dcotors

DOCTOR NAME	SCHEDULE ID	DAY	START TIME	END TIME	STATUS
ADITYA MISHRA	tcm11064_T1011	TUESDAY	10:am	11:am	available



SERACHING THE WHOLE LIST OF AVAILABLE DOCTORS IN THE HOSPITAL

What would you like to do? : 4

These are the avilable date and dcotors

DOCTOR NAME	SCHEDULE ID	DAY	START TIME	END TIME	STATUS
ADYASHA TIWARY	mem101184_M1112	MONDAY	11:am	12:am	available
ADYASHA TIWARY	mem101184_M0304	MONDAY	03:pm	04:pm	Booked
ADITYA MISHRA	tcm11064_I1011	TUESDAY	10:am	11:am	available



DOCTOR INTERFACE



USER INTERFACE FOR ANY DOCTOR IN THE HOSPITAL

What would you like to do? : 8

RETURNING TO THE MAIN INTERFACE

PATIENT APPOINTMENT SCHEDULING SYSTEM

- [1] PATIENT
- [2] DOCTOR
- [3] RECEPTIONIST
- [4] EXIT



CHOICES AVAILABLE TO ANY DOCTOR IN THE HOSPITAL

Enter your choice : 2

- [1] DISPLAY ALL PATIENT APPOINTMENTS
- [2] SEARCH PATIENT NAME
- [3] SEARCH PATIENT BY ID
- [4] UPDATE SCHEDULE
- [8] RETURN TO USER INTERFACE
- [9] EXIT

DISPLAYING THE WHOLE LIST OF PATIENT'S APPOINTMENTS IN THE HOSPITAL

What would you like to do? : 1

PATIENT'S ID	PATIENT'S NAME	DOCTOR'S NAME	APPOINTMENT DAY	APPOINTMENT TIME
PSSM79	SUMIT	ADYASHA	MONDAY	03:pm to 04:pm

SEARCHING A PARTICULAR PATIENT BY IT'S FIRST NAME

What would you like to do? : 2

Enter Patient First Name : SUMIT

PATIENT'S ID	PATIENT'S NAME	GENDER	DAY OF BIRTH	PHONE NO.
PSSM79	SUMIT SATAPATHY	MALE	23-11-2001	7978442590



SEARCHING A PARTICULAR PATIENT BY IT'S ID

What would you like to do? : 3

Enter Patient ID : PSSM79

PATIENT'S ID	PATIENT'S NAME	GENDER	DAY OF BIRTH	PHONE NO.
PSSM79	SUMIT SATAPATHY	MALE	23-11-2001	7978442590



UPDATING SCHEDULE OF ANY DOCTOR IN THE HOSPITAL

What would you like to do? : 4

FILL IN YOUR SCHDEULE INFORMATION

Enter your first name : SUMIT



RECEPTIONIST INTERFACE



USER INTERFACE FOR ANY RECEPTIONIST IN THE HOSPITAL

What would you like to do? : 8

RETURNING TO THE MAIN INTERFACE

PATIENT APPOINTMENT SCHEDULING SYSTEM

- [1] PATIENT
- [2] DOCTOR
- [3] RECEPTIONIST
- [4] EXIT



CHOICES AVAILABLE TO A RECEPTIONIST IN THE HOSPITAL

Enter your choice : 3

- [1] DISPLAY ALL PATIENT'S LIST
- [2] DISPLAY ALL DOCTOR'S LIST
- [3] SEARCH DOCTOR'S NAME
- [4] SEARCH AVAILABLE DOCTORS
- [5] SORT ALL PATIENT'S LIST BY FIRST NAME
- [6] UPDATE PATIENT'S PHONE NUMBER
- [7] DELETE PATIENT'S RECORD
- [8] RETURN TO USER INTERFACE
- [9] EXIT



DISPLAYING THE WHOLE LIST OF PATIENTS IN THE HOSPITAL

What would you like to do? : 1

PKMF85	KAVYA MUKHERJEE	FEMALE	15-08-1998	8542632108
PDGM82	DEBASHISH GOSWAMI	MALE	09-06-1988	8265197320
PDGM82	DEBASHISH GOSWAMI	MALE	09-06-1988	8265197320
PSSM79	SUMIT SATAPATHY	MALE	23-11-2001	7978442590

DISPLAYING THE WHOLE LIST OF DOCTORS IN THE HOSPITAL

What would you like to do? : 2

DOCTOR'S ID	DOCTOR'S NAME	GENDER	DAY OF BIRTH	PHONE NO.
men101184	ADYASHA TIWARY	FEMALE	17-09-1995	9462543785
tcm11064	ADITYA MISHRA	MALE	16-03-1992	8974562146



INFORMATION ABOUT A SPECIFIC DOCTOR BY IT'S FIRST NAME

What would you like to do? : 3

Please enter the doctor first name : ADYASHA

These are the available date and doctors

DOCTOR NAME	SCHEDULE ID	DAY	START TIME	END TIME	STATUS
mem101184	ADYASHA TIWARY	FEMALE	17-09-1995	9462543785	



INFORMATION ABOUT ALL AVAILABLE DOCTORS IN THE HOSPITAL

What would you like to do? : 4

These are the available date and doctors

DOCTOR NAME	SCHEDULE ID	DAY	START TIME	END TIME	STATUS
ADYASHA TIWARY	mem101184_M1112	MONDAY	11:am	12:am	available
ADYASHA TIWARY	mem101184_M0304	MONDAY	03:pm	04:pm	Booked
ADITYA MISHRA	tcm11064_I1011	TUESDAY	10:am	11:am	available



SORTING THE WHOLE LIST OF PATIENTS BY FIRST NAME IN ASCENDING ORDER

What would you like to do? : 5

//////////////////// BEFORE SORTING////////////////////

The Patient list Before Sorting in Asending order

PATIENT'S ID	PATIENT'S NAME	GENDER	DAY OF BIRTH	PHONE NO.
PKMF85	KAVYA MUKHERJEE	FEMALE	15-08-1998	8542632108
PDGM82	DEBASHISH GOSWAMI	MALE	09-06-1988	8265197320
PDGM82	DEBASHISH GOSWAMI	MALE	09-06-1988	8265197320
PSSM79	SUMIT SATAPATHY	MALE	23-11-2001	7978442590

//////////////////// AFTER SORTING////////////////////

The Patient list After Sorting in Asending order

PATIENT'S ID	PATIENT'S NAME	GENDER	DAY OF BIRTH	PHONE NO.
PDGM82	DEBASHISH GOSWAMI	MALE	09-06-1988	8265197320
PDGM82	DEBASHISH GOSWAMI	MALE	09-06-1988	8265197320
PKMF85	KAVYA MUKHERJEE	FEMALE	15-08-1998	8542632108
PSSM79	SUMIT SATAPATHY	MALE	23-11-2001	7978442590



UPDATING THE PHONE NUMBER OF ANY PATIENT

What would you like to do? : 6

PATIENT'S ID	PATIENT'S NAME	GENDER	DAY OF BIRTH	PHONE NO.
PDGM82	DEBASHISH GOSWAMI	MALE	09-06-1988	8265197320
PDGM82	DEBASHISH GOSWAMI	MALE	09-06-1988	8265197320
PKMF85	KAVYA MUKHERJEE	FEMALE	15-08-1998	8542632108
PSSM79	SUMIT SATAPATHY	MALE	23-11-2001	7978442590

Please enter the patiten Id : PKMF85

Please enter the new phone number : 7978556234

PHONE NUMBER IS UPDATED SUCCESSFULLY

PATIENT'S ID	PATIENT'S NAME	GENDER	DAY OF BIRTH	PHONE NO.
PDGM82	DEBASHISH GOSWAMI	MALE	09-06-1988	8265197320
PDGM82	DEBASHISH GOSWAMI	MALE	09-06-1988	8265197320
PKMF85	KAVYA MUKHERJEE	FEMALE	15-08-1998	7978556234
PSSM79	SUMIT SATAPATHY	MALE	23-11-2001	7978442590



DELETING THE RECORD OF ANY PATIENT IN THE HOSPITAL

What would you like to do? : 7

PATIENT'S ID	PATIENT'S NAME	GENDER	DAY OF BIRTH	PHONE NO.
PDGM82	DEBASHISH GOSWAMI	MALE	09-06-1988	8265197320
PDGM82	DEBASHISH GOSWAMI	MALE	09-06-1988	8265197320
PKMF85	KAVYA MUKHERJEE	FEMALE	15-08-1998	7978556234
PSSM79	SUMIT SATAPATHY	MALE	23-11-2001	7978442590

Please enter the patient Id : PDGM82
RECORD DELETED SUCCESSFULLY

PATIENT'S ID	PATIENT'S NAME	GENDER	DAY OF BIRTH	PHONE NO.
PDGM82	DEBASHISH GOSWAMI	MALE	09-06-1988	8265197320
PKMF85	KAVYA MUKHERJEE	FEMALE	15-08-1998	7978556234
PSSM79	SUMIT SATAPATHY	MALE	23-11-2001	7978442590



EXIT INTERFACE



APPLICATION OF EXIT

RETURNING TO THE MAIN INTERFACE

PATIENT APPOINTMENT SCHEDULING SYSTEM

[1] PATIENT
[2] DOCTOR
[3] RECEPTIONIST
[4] EXIT

Enter your choice : 4

THANK YOU FOR USING THIS PROGRAM !!!

Process exited after 3318 seconds with return value 0
Press any key to continue . . .



LIVE DEMO PRESENTATION



Using this HyperLink , we can go to our (PASS) files & access our software for live demo presentation .

https://drive.google.com/drive/u/0/folders/1d3zV29BpQo_xcoA3jMmIXF7As_uBGLYOq

THANK
YOU

